

App Development Project Report

Trekking Management App

1. Student Details

Name: Vivek Mittal

Roll Number: 23F1001475

Email: 23f1001475@ds.study.iitm.ac.in

About Me: I am currently pursuing BS degree in Data Science and Applications. I have completed my Bachelor of Science with Physics, Chemistry, and Mathematics from HNB Garhwal University. I have a strong interest in learning new technologies and expanding my technical skills. My interests include web application development, problem solving, and building practical software solutions. I am motivated to continuously enhance my technical skills and apply my knowledge to develop efficient and user friendly applications

2. Project Details

Project Title: SummitGo

Problem Statement:

The SummitGo Trekking Management System is a web based application developed to plan and manage trekking activities. Handling this process manually is often inefficient and prone to errors, Trek organizers must handle trek scheduling, participants registrations, staff assignments, payments, and booking records while ensuring accurate communications and data management. Manual processes or disconnected systems can lead to booking conflicts difficulty tracking trek availability, inefficient staff management, and limited access to reports. Participants also face challenges in finding suitable treks, making bookings, and monitoring their booking status. By automating these operations, the application improves efficiency, reduces manual errors, enhances data organization, and provides a seamless experience for both administrator and trekkers.

Approach:

The SummitGo app is built using [Vue.js](#) for the frontend, Flask for the backend, and SQLite for the database. The application uses REST APIs to connect the frontend and backend. It allows admin to manage treks, staff, and bookings, while trekkers can view available treks, make bookings, and track their trekking history as well. The system simplifies trekking management by automating tasks and storing all information at one place.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** for limited assistance with code suggestions and debugging support. It helped me improve code structure, provided guidance of Flask routes, Vue.js and helped with SQLAlchemy query construction.

The extent of AI/LLM usage is around **15–20%**, while the core system design, application logic, database design and feature implementation were completed independently.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.js	Frontend framework for building the user interface.
Bootstrap 5	Frontend styling and responsive design
JWT (Flask-JWT-Extended)	User authentication and role based access
Celery	Backend task processing (scheduled jobs and report
Redis	Caching frequently accessed data.
SQLite	Lightweight local database for storing user data
Vite	

	Development server and build tool for the Vue.js frontend
Jinja2	Template engine for generating email reports and reminders

5. Database Schema / ER Diagram

Tables:

1. **User** — stores user profile details (id, name, email, password, phone, role, active status)
2. **TrekRoute** — stores trek routes(id, route_name, location, difficulty, days_on_trail, altitude,description, image)
3. **Treks** — stores scheduled treks(id, trek_name, start_date, end_date, total_slots, available_slots, price, status)
4. **Booking** — stores trek registrations(id, trek_id, user_id, booking_date, booking_status, participants, total_amount)
5. **TrekStaffAssignment** — stores trek staff assignment details(id , trek_id, trek_Staff_id)

Relationships:

- One-to-Many → `User(role = trekker) - Booking`
- One-to-Many → `TrekRoute - Trek`
- One-to-Many → `Trek - Booking`
- One-to-Many → `Trek - TrekStaffAssignment`
- One-to-Many → `User(role = trek_staff) - TrekStaffAssignment`
- Many-to-Many → `User(role = trek_staff) - Trek`



6. API Resource Endpoints

Endpoint	Method	Description
/api/login	POST	Authenticate user and generate session
/api/register	POST	Register the User(trekker)
/api/admin/dashboard	GET	Fetch Bookings, Staffs and Trekkers details

<code>/api/admin/create_trek_route</code>	POST	Create the trek route
<code>/api/admin/all_treks</code>	GET	Fetch all the trek routes
<code>/api/admin/routes/<int:route_id></code>	GET	Fetch a trek route details
<code>/api/admin/routes/<int:route_id>/schedule</code>	POST	Schedule the trek
<code>/api/admin/scheduled_treks</code>	GET	Fetch all scheduled treks
<code>/api/admin/scheduled_trek/<int:trek_id></code>	GET	Scheduled trek details
<code>/api/admin/scheduled_treks/<int:trek_id>/update_trek</code>	PUT	Update scheduled trek
<code>/api/admin/scheduled_treks/<int:trek_id>/delete_trek</code>	POST	Delete Scheduled trek
<code>/api/admin/bookings</code>	GET	Fetch all bookings
<code>/api/admin/create_staff</code>	POST	Create trek staff
<code>/api/admin/treks/<int:trek_id>/assign_staff</code>	POST	Assign staff to a trek
<code>/api/admin/treks/<int:trek_id>/staff</code>	GET	Fetch staffs assigned to a trek
<code>/api/admin/assignments/<int:assignment_id></code>	POST	Remove assigned staff from a trek
<code>/api/admin/user_deactivate/<int:user_id></code>	POST	Deactivate(blacklist) user

<code>/api/admin/user_activate/<int:user_id></code>	POST	Reactivate(unblacklist) user
<code>/api/admin/user_delete/<int:user_id></code>	POST	Soft delete a user
<code>/api/admin/all_trekkers</code>	GET	Fetch all registered trekkers
<code>/api/admin/all_staffs</code>	GET	Fetch all registered staffs
<code>/api/admin/reports</code>	GET	Fetch trek. Booking and user details
<code>/api/staff/dashboard</code>	GET	Fetch assigned treks, and other dashboard data
<code>/api/staff/treks/<int:trek_id></code>	GET	Fetch details of assigned trek
<code>/api/staff/treks/<int:trek_id>/participants</code>	GET	Fetch trek participants and their details
<code>/api/staff/trek_update/<int:trek_id>/slots</code>	POST	Update trek slots and status
<code>/api/trekkers/dashboard</code>	GET	Fetch available treks, recent bookings, and other dashboard data
<code>/api/trekkers/treks/<int:trekID></code>	GET	Fetch trek details
<code>/api/trekkers/bookings</code>	POST	Book a trek
<code>/api/trekkers/bookings/<int:booking_id>/cancel</code>	POST	Cancel booked trek
<code>/api/trekkers/bookings</code>	GET	Fetch all the booked treks and their details
<code>/api/trekkers/history</code>	GET	Fetch the booking history
<code>/api/trekkers/history/export</code>	POST	Export export history as csv; export jobs are queued and processed asynchronously by celery workers.

`/api/trekkers/profile`

GET
PUT

Fetch and Update user(trekker) details

YAML API Definition File:

Included separately in the submission ZIP as `api.yaml`.

7. Architecture and Features

Architecture Overview:

- **app.py** – main Flask application entry point
 - **/models** – database models using SQLAlchemy
 - **controllers/auth** – Handles authentication and authorization, including user registration, login, logout and role based access control.
 - **controllers/admin** – Flask routes for admin operations.
 - **controllers/trekkers** – Flask routes for trekker functionalities.
 - **controllers/staff** – Flask routed for staff operations
 - **frontend/components** – Contains fronted components responsible for rendering user interface and handling client side interactions.
 - **/templates** – Jinja2 HTML templates for monthly repots and daily reminders.
 - **/static** – images and other frontend resources.
-

8. Video Presentation

Drive Link:

[SummitGo-Presentation-Video](#)

(Accessible to all with “View” permission.)
